Search in course

Search

English

B

Hide menu

- **Week 13 Information**
- **Virtualization**
- **Containers**
- **Week 13 Graded Activities**
  -

Graded Assignment: Week 13 Quiz

Submitted

  -

Programming Assignment: Machine Problem 12: Virtualization

. Duration: 6 hours6h

1. CS 498: Cloud Computing Applications
2. Week 13
3. Machine Problem 12: Virtualization

PreviousNext

Information:

After the due date, 10% will be deducted from your grade every day until you submit this. Learn more

Programming Assignment: Machine Problem 12: Virtualization

You have not submitted.

Deadline

Pass this assignment by May 11, 11:59 PM CDT

InstructionsMy submissionsDiscussions

1. Overview

Having developed one of the best image classification neural networks out there, it is now time to provide it as an online service and capture some market share. To do so, we set up and manage the infrastructure using Kubernetes and Docker containers over a cluster of nodes in this MP.

Essentially, you will start a web server interface for end-users to access machine learning services. When users send HTTP requests, your server is responsible for launching the corresponding ML jobs via Kubernetes. The autograder will act as an end-user, sending HTTP requests to your exposed server to grade the submissions.

Note: AWS Educate starter account will not work for this MP, so you need to have a personal AWS account.

This GitHub repository contains all the necessary files for this MP.

1.1 Recommended Setup: Use an EC2 Instance WE RECOMMEND DOING THIS ENTIRE MP ON AN EC2 INSTANCE. YOU CAN CREATE AN EC2 INSTANCE, CREATE DOCKER IMAGE IN THE SAME INSTANCE, CREATE CLUSTER FROM THAT INSTANCE AND DO ALL CLUSTER OPERATIONS FROM THAT INSTANCE. YOU NEED NOT USE YOUR PERSONAL COMPUTER. THIS WILL MAKE SETTING UP PERMISSIONS VERY EASY SINCE BOTH THE EC2 INSTANCE AND THE EKS WILL BE IN THE SAME AWS ACCOUNT. YOU MUST ALSO RUN THE WEB APPLICATION'S SERVER IN THE SAME INSTANCE AS THE INSTANCE WILL HAVE CLUSTER CONFIGURATION.

2. Requirements

Note: Please use region -> us-east-1 for your deployment.

You need an AWS account with some free credit available and will work on EKS, EC2, Kubernetes, and Docker. AWS Educate accounts will not work as these accounts cannot

create the network resources (EKS) needed for this assignment. Also, you need to be familiar with one of the following programming languages for implementing your server and interacting with Kubernetes: Python 3 / Javascript / Java / Go. While we will attempt to help out irrespective of your chosen language, we can best assist with Python.

3. Procedure

3.1 Containerization

We have provided the image classification neural network implementation with two versions - (classify.py). This file contains the code for both the versions. The first is a simple feed-forward neural network that you will use to market your product, providing it as a **free service**. The second is your **premium service** - a convolutional neural network. The provided implementation retrieves the dataset from the internet, trains the model, and then performs testing/classification on it. The 'DATASET' can be *mnist* or *kmnist,* and the 'TYPE' can be ff or *cnn,* both passed in using environment variables. Here, you only need to know how to run classify.py, not worry about its implementation.

First, you have to dockerize the classification script to run it on your cluster. You need to create a docker image for the classifier. You must pass the environment variables (DATASET and TYPE) from the Dockerfile. This is required to run the container for your image after it is built. We have also provided requirements.txt, which specifies the python packages that your docker image needs. Once you have created the image, you should test that it works when you run it in a docker container.

Note 3.1.1: the provided requirements.txt file was developed against the Amazon Linux 2023 AMI with x86 architecture. You may need to modify the values if building locally rather than on ec2.

Note 3.1.2: If you are building locally (and not on Amazon Linux 2023 AMI EC2 instance) on a different architecture (such as ARM-based Macs or other Linux distributions), you may need to modify dependencies. To ensure compatibility across different platforms, consider using Docker Buildx for multi-architecture builds.

There are 2 scripts that need to run inside your dockerfile. The data_preload.py file which will download the training data. Then train.py needs to be run for each model and for each dataset in order to save pretrained models into your docker image.

We provided a template for the Dockerfile which you need to complete. You can refer the template and create the Dockerfile. The template is only for reference, you can create the Dockerfile in any other way as you wish as long as it works.

Docker resource

The expected output of successfully running a docker container is similar to the following:

1

2

3

4

5

6

7

8

9

10

11

dataset: mnist

Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz to ./data/MNIST/raw/train-images-idx3-ubyte.gz

100.1%Extracting ./data/MNIST/raw/train-images-idx3-ubyte.gz to ./data/MNIST/raw

...

Processing...

Done!

Epoch [1/5], Step [100/600], Loss: 0.2846

Epoch [1/5], Step [200/600], Loss: 0.3859

Epoch [1/5], Step [300/600], Loss: 0.1448

...

Accuracy of the network on the 10K test images: 98 %

3.2 Pushing the Image to a repository

Now that the container image is ready, you need to push the image to a repository/registry. You can either use **AWS ECR** or **Dockerhub** *(recommended for ease-of-use)* for this

purpose. The worker nodes in the kubernetes cluster will pull the image from the registry and use them for the job.

Dockerhub reference

AWS ECR documentation

3.3 Minikube guide -- for local cluster testing

Before implementing your solution on an AWS EKS cluster it is recommended to test against a local cluster that has been created using Minikube. This step can be skipped but it may help iterate faster running locally.

Follow the appropriate installation instructions for minikube here https://minikube.sigs.k8s.io/docs/start/ and kubectl here https://kubernetes.io/docs/tasks/tools/

Some useful minikube commands are:

1

2

3

4

5

6

7

8

start the minikube cluster

minikube start

load your docker image onto the cluster

minikube image load mp12_image

view the minikube dashboard

view minikube dashboard `minikube dashboard`

Note 3.3.1: you will need to add imagePullPolicy: IfNotPresent to your k8s resource yaml when using minikube.

3.4 AWS EKS / Kubernetes Setup

Prepare a host machine through which you will create and manage your EKS cluster. Technically, you can use your personal computer or machine to create and manage EKS cluster. But, we highly recommend you to use an EC2 instance instead of your own machine. You can read the note in the above section.

You need to install aws cli, kubectl and eksctl in order to create and manage the EKS cluster.

Resources:

1. EKS user guide
2. kubernetes documentation
3. Video reference for EKS setup
4. kubectl cheatsheet
5. Creating a cluster with eksctl

Note 3.4.1: For your EC2 instance to have permissions to create an EKS cluster, you need to either set up the credentials using "aws configure" command (you need to give access key for your account) or use an IAM role for the instance will all the required policies to create a cluster.

Note 3.4.2: We highly recommend to use a "ClusterConfig" type YAML file to create the cluster using eksctl instead of configuring the cluster yourself. Below is the yaml file you can use to create the cluster. The below file contains the configuration for the cluster which will have 2 nodes of type (t2.medium). We recommend you to use t2.medium instances which will have 4 CPUs (2 CPUs in each node) which is required for this MP.

1

2

3

4

5

```
6
7
8
9
10
11
12
13
14
15
16
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
   name: mp12-cluster
   region: us-east-1

availabilityZones:
  - us-east-1a
  - us-east-1b

nodeGroups:
 - name: ng-1
   instanceType: t2.medium
   desiredCapacity: 2
```

privateNetworking: true

3.5 Resource Provisioning

As you will be hosting both free and premium services, you must provision your cluster appropriately to run the jobs. You do not want the free service to take up all the available hardware resources. To do so, you will provision at most **2** CPUs out of the total 4 (t2.medium has two cores) for your free service. We do this by using the Kubernetes namespace to create a virtual cluster. You can create a **"free-service"** namespace and create a quota for this namespace by using a ResourceQuota yaml file. Pods in this namespace cannot request and use more than **2** CPUs. For the premium service jobs, you must use the **"default"** namespace. We want to ensure that at any time, only two free service pods are running (the other free service pods should be queued up). However, there can be any number of premium pods running.

Reference: Configure Memory and CPU Quotas for a Namespace

There is one more constraint which you must set up. Each of the jobs should request and have a limit of **0.9** CPU. This requirement applies to both free and premium service as well. You must specify this limit in the Job YAML files. More about Job yaml file in the next section.

Note 3.5.1: Premium service pods should execute under the "default" namespace and the free ones under the "free-service" namespace.

3.6 Create Job YAML files

To launch or deploy kubernetes jobs from the server code, you need to create 2 "Job" type yaml files - one for free service and the other for premium service jobs. These job yaml files must have the following 3 details apart from other required information:

1. Image name: the image reference/name in either Dockerhub or ECR. Kubernetes will use this information to the pull the image from the corresponding registry

2. Environment variables: you need to specify the appropriate environment variables (DATASET and TYPE) in the yaml files. These variable values will be different in both the yaml files

3. Resource limits: as mentioned in the above section, you must ensure that each job can request and use at most 0.9 CPU. This applies to both free and premium jobs. This limit must be specified in the yaml files.

You can follow the official documentation of Kubernetes to learn how to deploy your job.

Here is an example of how to set environment variables in a Kubernetes YAML file (note that you will not be using “kind: Pod” as described in the following guide. This is just an example of environment variables usage in YAML.)

Note 3.6.1: You should only create Job yaml files. You need not create or start any jobs or pods on your own. When you make a submission, the autograder will send requests to your server which will then create jobs. More about this server code in the next section.

Note 3.6.2: Kubernetes will only create one Job per a single name. We cannot create multiple jobs with the same name. Therefore, you can use "generateName" field instead of "name" field in the job yaml file. With "generateName", kubernetes will automatically generate a new name and append to the value you provided in the "generateName" field so that every job will have a unique name. You can refer to this document for more details.

3.7 Exposing Image Classification

Now that the cluster is ready to use, you will build a server that accepts image classification requests and the necessary configurations from the autograder to verify your implementation. You will implement the server on the machine you used to set up the EKS, which has the Kubernetes configuration. We recommend running the server on an EC2 instance, not exposing your local network to the autograder. The main part is programmatically interacting with Kubernetes, which you can achieve by using one of these client libraries.

We provide you with the template code which you can use.

The two APIs you will need are **list_pod_for_all_namespaces()** and **create_namespaced_job()**.

You will be implementing the following API's:

1. Free Service - Create a container (job) with the feed-forward neural network within the free-service namespace

- HTTP POST: /img-classification/free
- BODY: {"dataset":"mnist"} (even though this looks like JSON string, the content type might not be application/json, so parse the request body accordingly). Since we will always use mnist in free service requests, you can simply hardcode the dataset name.
- The response status code should be 200 **if the job has been successfully created**. Based on your namespace configuration, only two free service jobs should be running at a time.

2. Premium Service - Create a container (job) with the convolutional neural network within the default namespace

- HTTP POST: /img-classification/premium
- BODY: {"dataset":"kmnist"} (even though this looks like JSON string, the content type might not be application/json, so parse the request body accordingly) Since we will always use kmnist in premium service requests, you can simply hardcode the dataset name.
- The response status code should be 200. You are hypothetically getting paid for this service.

3. Configuration - Sends a snapshot of the current outlook of your Kubernetes cluster by returning information about all pods across all namespaces.

- HTTP GET: /config
- Response status code should be 200, and the body should be valid JSON. The body contains all the pods, including those created by Kubernetes that are currently executing with some specifications. The following are the specifications:
- BODY: { "pods": [pod1, pod2 ...]}
    - pods : a list of pods
    - a pod: {"node" : "node on which the pod is executing", "ip" : "ip address of the pod", "namespace" : "namespace of the node", "name" : "name of the pod", "status":"status of the pod"}

For example:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18


```
{
  "pods": [
    {
      "node": "ip-192-168-123-7.ec2.internal",
      "ip": "192.168.125.2",
      "namespace": "default",
      "name": "mnist-deployment-6dc644dd6d-grpfd",
      "status": "Succeeded"
    },
    {
      "node": "ip-192-168-123-7.ec2.internal",
      "ip": "192.168.125.2",
      "namespace": "default",
      "name": "mnist-deployment-6dc644dd6d-grpfd",
      "status": "Succeeded"
    }
```

]

}

The following are the object mappings for the python code:

1

2

3

4

5

6

7

{

"name": pod.metadata.name,

"ip": pod.status.pod_ip,

"namespace":pod.metadata.namespace,

"node":pod.spec.node_name,

"status": pod.status.phase

}

4. Autograder

Autograder does the following tests to test your submission:

Test 1: Make a GET request and check whether there are correct number of system pods

Test 2: Make multiple POST requests to /img-classification/free and check if there correct number of free service pods

Test 3: Make multiple POST requests to /img-classification/premium and check if there correct number of premium service pods (default namespace)

Test 4: Make a GET call and check if the free service and default pods are successfully completed.

Note: the first time an image is run on your cluster it needs to be pulled into the cluster and cached. This will cause the initial run to take longer than expected while its downloading the pod and as such you may fail the autograder. To avoid this be sure to run your image on your cluster at least once so it is cached before the autograder runs.

5. Final Result / Submission

Virtualization helped us effectively provision our hardware resources and achieve isolation between different executions. Moreover, we could easily deploy our neural network in any of our nodes without worrying about the environment, dependencies, etc.

Now that your service is ready, it is time to submit it to the autograder. Please ensure that you do not have any additional pods/jobs running other than the Kubernetes ones before submitting them. Add the necessary info in the payload section in the test.py file and execute the program. It takes about 20 seconds to run all of the test cases on your infrastructure. If all the test cases pass, you will see your grade on Coursera.

You can find the test.py submission script in the github repository.

Note: Please make sure that you delete all jobs and all pods in all namespaces (free-service and default) before making a submission. Otherwise, you will not pass the autograder tests.

6. Resource clean up

After submitting the MP successfully, make sure you delete all the resources including EC2 instance, EKS cluster, CloudFormation stacks, and VPC.

If you receive an error deleting one of the CloudFormation stack, then you need to delete the VPC and then comeback and delete the stack again.

7. Useful commands

You can refer to the kubernetes cheatsheet for all the commands, but here are few which might be useful in this MP. You can read about what each command does before using them:

1

2

3

4

5

6

7

8

9

10

kubectl get nodes

kubectl get pods -A

kubectl describe nodes

kubectl describe pod --namespace=free-service

kubectl top nodes

kubectl top pods

kubectl delete --all all --namespace=free-service

kubectl delete --all all --namespace=default

kubectl get quota --namespace=free-service

kubectl delete quota/<quota name> --namespace=free-service

How to submit

Copy the token below and run the submission script included in the assignment download. When prompted, use your email address **fridkis2@illinois.edu**.

TCE5RgCzX1QIwu2n

This token expires in 24 minutes

Generate new token

Your submission token is unique to you and should not be shared with anyone. You may submit as many times as you like.

Like

Dislike

Report an issue